

Chapter 12

Transformers

Chapter 10 introduced convolutional networks, which are specialized for processing data that lie on a regular grid. They are particularly suited to processing images, which have a very large number of input variables, precluding the use of fully connected networks. Each layer of a convolutional network employs parameter sharing so that local image patches are processed similarly at every position in the image.

This chapter introduces transformers. These were initially targeted at natural language processing (NLP) problems, where the network input is a series of high-dimensional embeddings representing words or word fragments. Language datasets share some of the characteristics of image data. The number of input variables can be very large, and the statistics are similar at every position; it's not sensible to re-learn the meaning of the word `dog` at every possible position in a body of text. However, language datasets have the complication that text sequences vary in length, and unlike images, there is no easy way to resize them.

12.1 Processing text data

To motivate the transformer, consider the following passage:

The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food and service.

The goal is to design a network to process this text into a representation suitable for downstream tasks. For example, it might be used to classify the review as positive or negative or to answer questions such as “Does the restaurant serve steak?”.

We can make three immediate observations. First, the encoded input can be surprisingly large. In this case, each of the 37 words might be represented by an embedding vector of length 1024, so the encoded input would be of length $37 \times 1024 = 37888$ even for this small passage. A more realistically sized body of text might have hundreds or even thousands of words, so fully connected neural networks are impractical.

Second, one of the defining characteristics of NLP problems is that each input (one or more sentences) is of a different length; hence, it's not even obvious how to apply a fully connected network. These observations suggest that the network should share parameters across words at different input positions, similarly to how convolutional networks share parameters across different image positions.

Third, language is ambiguous; it is unclear from the syntax alone that the pronoun *it* refers to the restaurant and not to the ham sandwich. To understand the text, the word *it* should somehow be connected to the word *restaurant*. In the parlance of transformers, the former word should pay *attention* to the latter. This implies that there must be connections between the words and that the strength of these connections will depend on the words themselves. Moreover, these connections need to extend across large text spans. For example, the word *their* in the last sentence also refers to the restaurant.

12.2 Dot-product self-attention

The previous section argued that a model for processing text will (i) use parameter sharing to cope with long input passages of differing lengths and (ii) contain connections between word representations that depend on the words themselves. The transformer acquires both properties by using *dot-product self-attention*.

A standard neural network layer $\mathbf{f}[\mathbf{x}]$, takes a $D \times 1$ input $\mathbf{x}$ and applies a linear transformation followed by an activation function like a ReLU, so:

$$\mathbf{f}[\mathbf{x}] = \text{ReLU}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}], \quad (12.1)$$

where $\boldsymbol{\beta}$ contains the biases, and $\boldsymbol{\Omega}$ contains the weights.

A self-attention block $\mathbf{sa}[\bullet]$ takes N inputs $\mathbf{x}_1, \dots, \mathbf{x}_N$, each of dimension $D \times 1$, and returns N outputs, each of which is also of size $D \times 1$. In the context of NLP, each input represents a word or word fragment. First, a set of *values* are computed for each input:

$$\mathbf{v}_m = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v \mathbf{x}_m, \quad (12.2)$$

where $\boldsymbol{\beta}_v \in \mathbb{R}^{D \times 1}$ and $\boldsymbol{\Omega}_v \in \mathbb{R}^{D \times D}$ represent biases and weights, respectively.

Then the n^{th} output $\mathbf{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N]$ is a weighted sum of all the values $\mathbf{v}_1, \dots, \mathbf{v}_N$:

$$\mathbf{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N] = \sum_{m=1}^N a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m. \quad (12.3)$$

The scalar weight $a[\mathbf{x}_m, \mathbf{x}_n]$ is the *attention* that the n^{th} output pays to input $\mathbf{x}_m$. The N weights $a[\bullet, \mathbf{x}_n]$ are non-negative and sum to one. Hence, self-attention can be thought of as *routing* the values in different proportions to create each output (figure 12.1).

The following sections examine dot-product self-attention in more detail. First, we consider the computation of the values and their subsequent weighting (equation 12.3). Then we describe how to compute the attention weights $a[\mathbf{x}_m, \mathbf{x}_n]$ themselves.

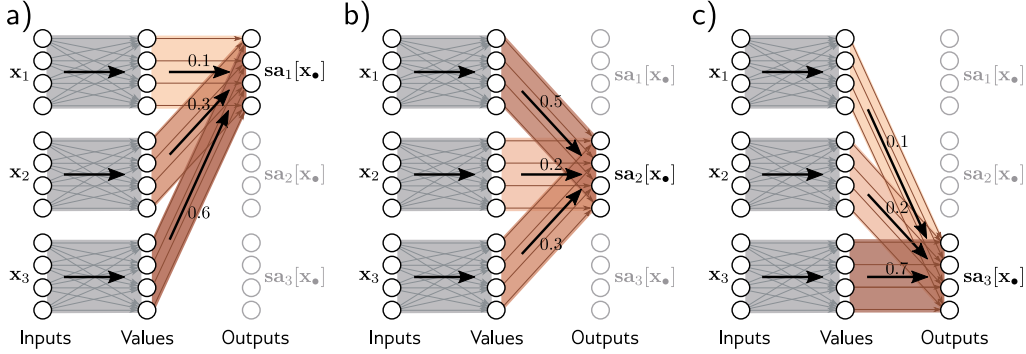


Figure 12.1 Self-attention as routing. The self-attention mechanism takes N inputs $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (here $N = 3$ and $D = 4$) and processes each separately to compute N value vectors. The n^{th} output $\mathbf{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N]$ (written as $\mathbf{sa}_n[\mathbf{x}_\bullet]$ for short) is then computed as a weighted sum of the N value vectors, where the weights are positive and sum to one. a) Output $\mathbf{sa}_1[\mathbf{x}_\bullet]$ is computed as $a[\mathbf{x}_1, \mathbf{x}_1] = 0.1$ times the first value vector, $a[\mathbf{x}_2, \mathbf{x}_1] = 0.3$ times the second value vector, and $a[\mathbf{x}_3, \mathbf{x}_1] = 0.6$ times the third value vector. b) Output $\mathbf{sa}_2[\mathbf{x}_\bullet]$ is computed in the same way, but this time with weights of 0.5, 0.2, and 0.3. c) The weighting for output $\mathbf{sa}_3[\mathbf{x}_\bullet]$ is different again. Each output can hence be thought of as a different routing of the N values.

12.2.1 Computing and weighting values

Equation 12.2 shows that the same weights $\mathbf{\Omega}_v \in \mathbb{R}^{D \times D}$ and biases $\beta_v \in \mathbb{R}^D$ are applied to each input $\mathbf{x}_\bullet \in \mathbb{R}^D$. This computation scales linearly with the sequence length N , so it needs fewer parameters than a fully connected network relating all DN inputs to all DN values. In fact, the value computation can be viewed as a sparse matrix operation with shared parameters that relates these DN quantities (figure 12.2b).

The attention weights $a[\mathbf{x}_m, \mathbf{x}_n]$ combine the values from different inputs. They are also sparse since there is only one weight for each ordered pair of inputs $(\mathbf{x}_m, \mathbf{x}_n)$, regardless of the size of these inputs (figure 12.2c). It follows that the number of attention weights has a quadratic dependence on the sequence length N , but is independent of the length D of each input.

12.2.2 Computing attention weights

In the previous section, we saw that the outputs result from two chained linear transformations; the value vectors $\beta_v + \mathbf{\Omega}_v \mathbf{x}_m$ are computed independently for each input $\mathbf{x}_m$, and these vectors are combined linearly by the attention weights $a[\mathbf{x}_m, \mathbf{x}_n]$. However, the overall self-attention computation is *nonlinear*. As we'll see shortly, the attention weights are themselves nonlinear functions of the input. This is an example of a *hyper-network*, where one network branch computes the weights of another. To compute the

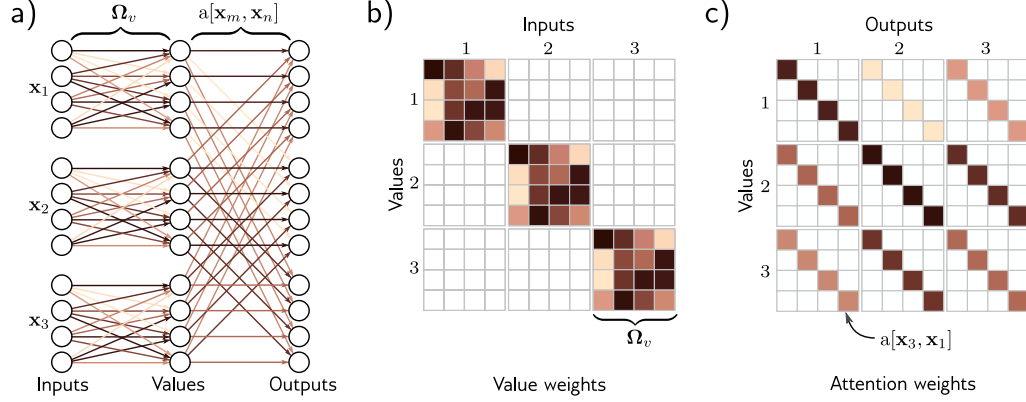


Figure 12.2 Self-attention for $N=3$ inputs $\mathbf{x}_n$, each with dimension $D=4$. a) Each input $\mathbf{x}_m$ is operated on independently by the same weights Ω_v (same color equals same weight) and biases β_v (not shown) to form the values $\beta_v + \Omega_v \mathbf{x}_m$. Each output is a linear combination of the values, with the attention weight $a[\mathbf{x}_m, \mathbf{x}_n]$ defining the contribution of the m^{th} value to the n^{th} output. b) Matrix showing block sparsity of linear transformation Ω_v between inputs and values. c) Matrix showing sparsity of attention weights relating values and outputs.

attention, we apply two more linear transformations to the inputs:

$$\begin{aligned} \mathbf{q}_n &= \beta_q + \Omega_q \mathbf{x}_n \\ \mathbf{k}_m &= \beta_k + \Omega_k \mathbf{x}_m, \end{aligned} \quad (12.4)$$

where $\{\mathbf{q}_n\}$ and $\{\mathbf{k}_m\}$ are termed *queries* and *keys*, respectively. Then we compute **dot products** between the queries and keys and pass the results through a softmax function:

$$\begin{aligned} a[\mathbf{x}_m, \mathbf{x}_n] &= \text{softmax}_m [\mathbf{k}_\bullet^T \mathbf{q}_n] \\ &= \frac{\exp [\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^N \exp [\mathbf{k}_{m'}^T \mathbf{q}_n]}, \end{aligned} \quad (12.5)$$

so for each $\mathbf{x}_n$, they are positive and sum to one (figure 12.3). For obvious reasons, this is known as *dot-product self-attention*.

The names “queries” and “keys” were inherited from the field of information retrieval and have the following interpretation: the dot product operation returns a measure of similarity between its inputs, so the weights $a[\mathbf{x}_\bullet, \mathbf{x}_n]$ depend on the relative similarities between the n^{th} query and all of the keys. The softmax function means that the key vectors “compete” with one another to contribute to the final result. The queries and keys must have the same dimensions. However, these can differ from the dimension of

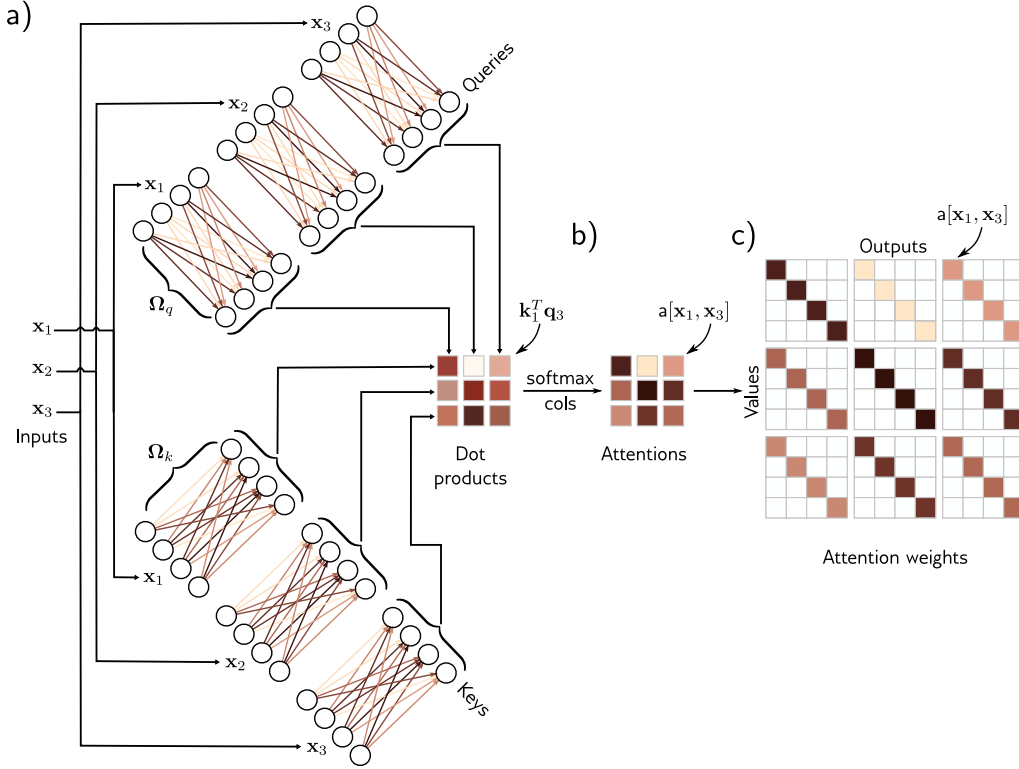


Figure 12.3 Computing attention weights. a) Query vectors $\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$ and key vectors $\mathbf{k}_n = \beta_k + \Omega_k \mathbf{x}_n$ are computed for each input $\mathbf{x}_n$. b) The dot products between each query and the three keys are passed through a softmax function to form non-negative attentions that sum to one. c) These route the value vectors (figure 12.1) via the sparse matrix from figure 12.2c.

the values, which is usually the same size as the input, so the representation doesn't change size.

Problems 12.1–12.2

12.2.3 Self-attention summary

The n^{th} output is a weighted sum of the same linear transformation $\mathbf{v}_\bullet = \beta_v + \Omega_v \mathbf{x}_\bullet$ applied to all of the inputs, where these attention weights are positive and sum to one. The weights depend on a measure of similarity between input $\mathbf{x}_n$ and the other inputs. There is no activation function, but the mechanism is nonlinear due to the dot-product and a softmax operation used to compute the attention weights.

Note that this mechanism fulfills the initial requirements. First, there is a single shared set of parameters $\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$. This is independent of the

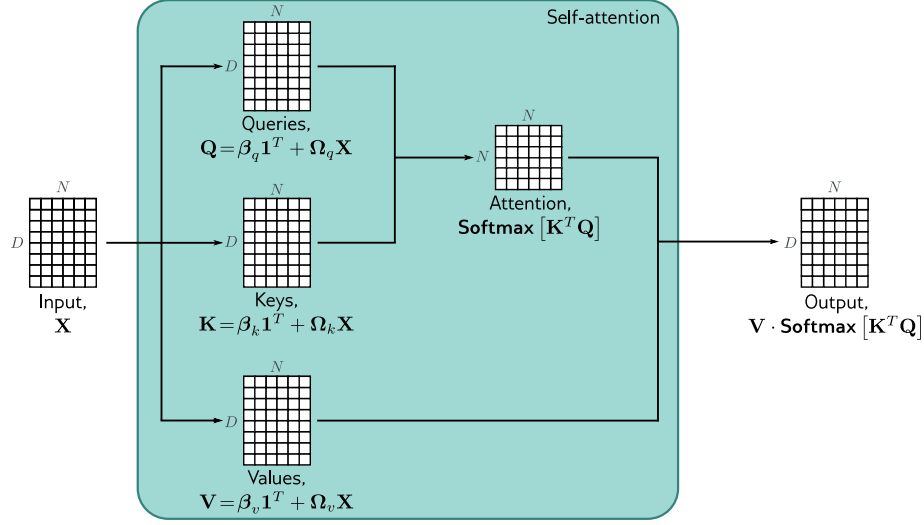


Figure 12.4 Self-attention in matrix form. Self-attention can be implemented efficiently if we store the N input vectors $\mathbf{x}_n$ in the columns of the $D \times N$ matrix $\mathbf{X}$. The input $\mathbf{X}$ is operated on separately by the query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$. The dot products are then computed using matrix multiplication, and a softmax operation is applied independently to each column of the resulting matrix to calculate the attentions. Finally, the values are post-multiplied by the attentions to create an output of the same size as the input.

number of inputs N , so the network can be applied to different sequence lengths. Second, there are connections between the inputs (words), and the strength of these connections depends on the inputs themselves via the attention weights.

12.2.4 Matrix form

The above computation can be written in a compact form if the N inputs $\mathbf{x}_n$ form the columns of the $D \times N$ matrix $\mathbf{X}$. The values, queries, and keys can be computed as:

$$\begin{aligned} \mathbf{V}[\mathbf{X}] &= \beta_v \mathbf{1}^T + \Omega_v \mathbf{X} \\ \mathbf{Q}[\mathbf{X}] &= \beta_q \mathbf{1}^T + \Omega_q \mathbf{X} \\ \mathbf{K}[\mathbf{X}] &= \beta_k \mathbf{1}^T + \Omega_k \mathbf{X}, \end{aligned} \tag{12.6}$$

where $\mathbf{1}$ is an $N \times 1$ vector containing ones. The self-attention computation is then:

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \text{Softmax}[\mathbf{K}[\mathbf{X}]^T \mathbf{Q}[\mathbf{X}]], \tag{12.7}$$

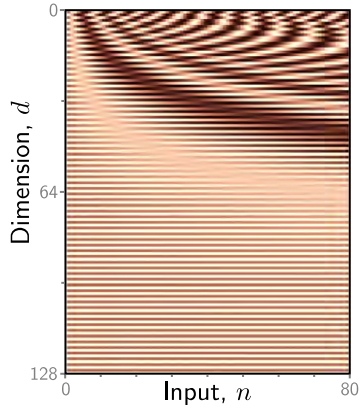


Figure 12.5 Positional encodings. The self-attention architecture is equivariant to permutations of the inputs. To ensure that inputs at different positions are treated differently, a positional encoding matrix $\mathbf{\Pi}$ can be added to the data matrix. Each column is different, so the positions can be distinguished. Here, the position encodings use a predefined procedural sinusoidal pattern (which can be extended to larger values of N if necessary). However, in other cases, they are learned.

where the function $\text{Softmax}[\bullet]$ takes a matrix and performs the softmax operation independently on each of its columns (figure 12.4). In this formulation, we have explicitly included the dependence of the values, queries, and keys on the input $\mathbf{X}$ to emphasize that self-attention computes a kind of triple product based on the inputs. However, from now on, we will drop this dependence and just write:

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]. \quad (12.8)$$

[Notebook 12.1](#)
[Self-attention](#)

12.3 Extensions to dot-product self-attention

In the previous section, we described self-attention. Here, we introduce three extensions that are almost always used in practice.

12.3.1 Positional encoding

Observant readers will have noticed that the self-attention mechanism overlooks important information: the computation does not take into account the order of the inputs $\mathbf{x}_n$. More precisely, it is equivariant with respect to input permutations. However, order *is* important when the inputs correspond to the words in a sentence. The sentence [The woman ate the raccoon](#) has a different meaning than [The raccoon ate the woman](#). There are two main approaches to incorporating position information.

[Problem 12.3](#)

Absolute positional encodings: A matrix $\mathbf{\Pi}$ is added to the input $\mathbf{X}$ that encodes positional information (figure 12.5). Each column of $\mathbf{\Pi}$ is unique and hence contains information about the absolute position in the input sequence. This matrix can be chosen by hand or learned. It may be added to the network inputs or at every network layer. Sometimes it is added to $\mathbf{X}$ in the computation of the queries and keys but not to the values.

Relative positional encodings: The input to a self-attention mechanism may be an entire sentence, many sentences, or just a fragment of a sentence, and the absolute position of a word is much less important than the relative position between two words. Of course, this can be recovered if the system knows the absolute position of both, but relative positional encodings encode this information directly. Each element of the attention matrix corresponds to a particular offset between key position a and query position b . Relative positional encodings learn a parameter $\pi_{a,b}$ for each offset and use this to modify the attention matrix by adding these values, multiplying by them, or using them to alter the attention matrix in some other way.

12.3.2 Scaled dot-product self-attention

Problem 12.4

The dot products in the attention computation can have large magnitudes and move the arguments to the softmax function into a region where the largest value completely dominates. Small changes to the inputs to the softmax function now have little effect on the output (i.e., the gradients are very small), making the model difficult to train. To prevent this, the dot products are scaled by the square root of the dimension D_q of the queries and keys (i.e., the number of rows in $\mathbf{\Omega}_q$ and $\mathbf{\Omega}_k$, which must be the same):

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \mathbf{Softmax} \left[\frac{\mathbf{K}^T \mathbf{Q}}{\sqrt{D_q}} \right]. \quad (12.9)$$

This is known as *scaled dot-product self-attention*.

12.3.3 Multiple heads

Multiple self-attention mechanisms are usually applied in parallel, and this is known as *multi-head self-attention*. Now H different sets of values, keys, and queries are computed:

$$\begin{aligned} \mathbf{V}_h &= \beta_{vh} \mathbf{1}^T + \mathbf{\Omega}_{vh} \mathbf{X} \\ \mathbf{Q}_h &= \beta_{qh} \mathbf{1}^T + \mathbf{\Omega}_{qh} \mathbf{X} \\ \mathbf{K}_h &= \beta_{kh} \mathbf{1}^T + \mathbf{\Omega}_{kh} \mathbf{X}. \end{aligned} \quad (12.10)$$

The h^{th} self-attention mechanism or *head* can be written as:

$$\mathbf{Sa}_h[\mathbf{X}] = \mathbf{V}_h \cdot \mathbf{Softmax} \left[\frac{\mathbf{K}_h^T \mathbf{Q}_h}{\sqrt{D_q}} \right], \quad (12.11)$$

where we have different parameters $\{\beta_{vh}, \mathbf{\Omega}_{vh}\}$, $\{\beta_{qh}, \mathbf{\Omega}_{qh}\}$, and $\{\beta_{kh}, \mathbf{\Omega}_{kh}\}$ for each head. Typically, if the dimension of the inputs $\mathbf{x}_m$ is D and there are H heads, the values, queries, and keys will all be of size D/H , as this allows for an efficient implementation. The outputs of these self-attention mechanisms are vertically concatenated, and another linear transform $\mathbf{\Omega}_c$ is applied to combine them (figure 12.6):

Problem 12.5

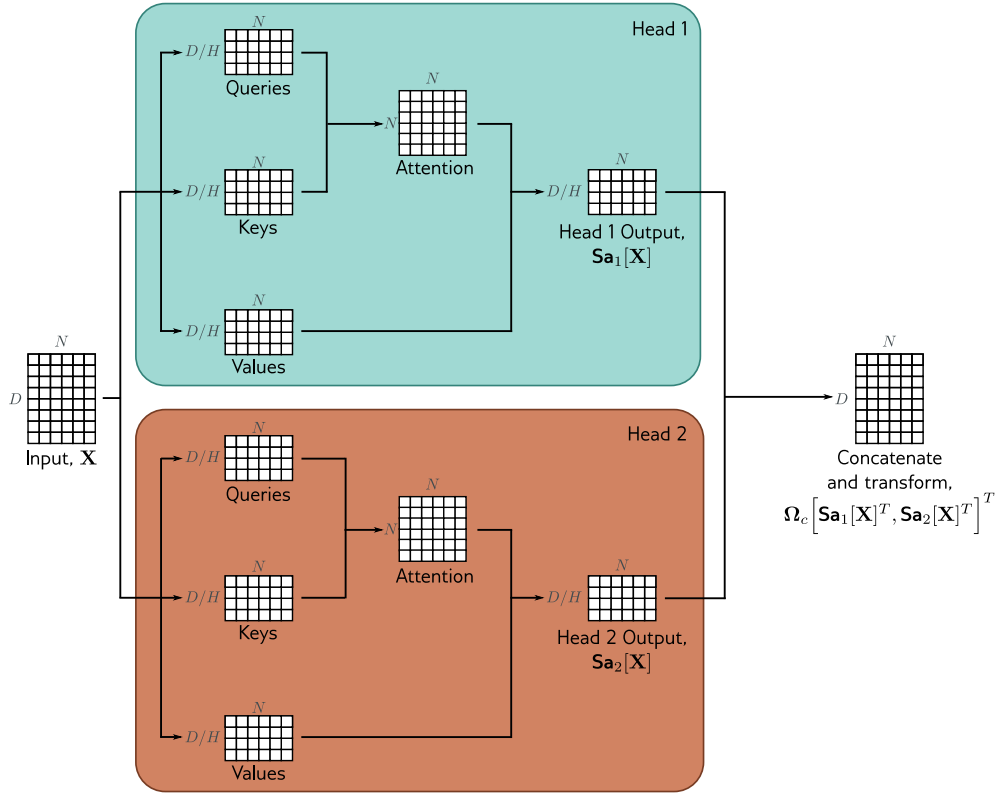


Figure 12.6 Multi-head self-attention. Self-attention occurs in parallel across multiple “heads.” Each has its own queries, keys, and values. Here two heads are depicted, in the cyan and orange boxes, respectively. The outputs are vertically concatenated, and another linear transformation Ω_c is used to recombine them.

$$\text{MhSa}[\mathbf{X}] = \Omega_c \left[\mathbf{Sa}_1[\mathbf{X}]^T, \mathbf{Sa}_2[\mathbf{X}]^T, \dots, \mathbf{Sa}_H[\mathbf{X}]^T \right]^T. \quad (12.12)$$

Multiple heads seem to be necessary to make self-attention work well. It has been speculated that they make the self-attention network more robust to bad initializations.

Notebook 12.2
Multi-head
self-attention

12.4 Transformer layers

Self-attention is just one part of a larger *transformer* layer. This consists of a multi-head self-attention unit (which allows the word representations to interact with each

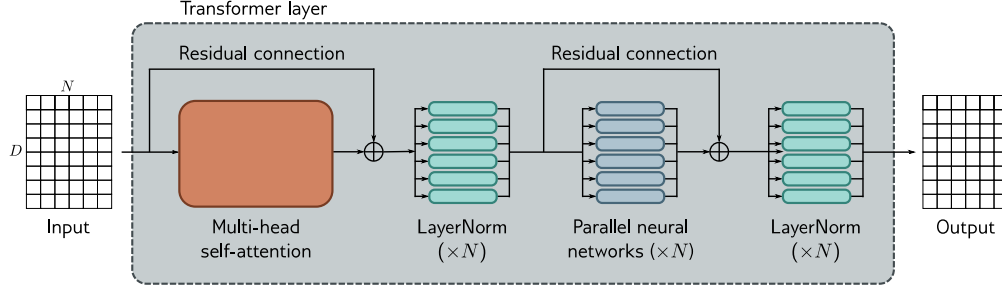


Figure 12.7 Transformer layer. The input consists of a $D \times N$ matrix containing the D -dimensional word embeddings for each of the N input tokens. The output is a matrix of the same size. The transformer layer consists of a series of operations. First, there is a multi-head attention block, allowing the word embeddings to interact with one another. This forms the processing of a residual block, so the inputs are added back to the output. Second, a LayerNorm operation is applied separately to each embedding. Third, there is a second residual layer where the same fully connected neural network is applied separately to each of the N word representations (columns). Finally, LayerNorm is applied again.

other) followed by a fully connected network $\mathbf{mlp}[\mathbf{x}_n]$ (that operates separately on each word). Both units are residual networks (i.e., their output is added back to the original input). In addition, it is typical to add a LayerNorm operation after both the self-attention and fully connected networks. This is similar to BatchNorm but normalizes each embedding in each batch element separately using statistics calculated across its D embedding dimensions (section 11.4 and figure 11.14). The complete layer can be described by the following series of operations (figure 12.7):

$$\begin{aligned}
 \mathbf{X} &\leftarrow \mathbf{X} + \mathbf{MhSa}[\mathbf{X}] \\
 \mathbf{X} &\leftarrow \mathbf{LayerNorm}[\mathbf{X}] \\
 \mathbf{x}_n &\leftarrow \mathbf{x}_n + \mathbf{mlp}[\mathbf{x}_n] & \forall n \in \{1, \dots, N\} \\
 \mathbf{X} &\leftarrow \mathbf{LayerNorm}[\mathbf{X}],
 \end{aligned} \tag{12.13}$$

where the column vectors $\mathbf{x}_n$ are separately taken from the full data matrix $\mathbf{X}$. In a real network, the data passes through a series of these transformer layers.

12.5 Transformers for natural language processing

The previous section described the transformer layer. This section describes how it is used in natural language processing (NLP) tasks. A typical NLP pipeline starts with a *tokenizer* that splits the text into words or word fragments. Then each of these tokens

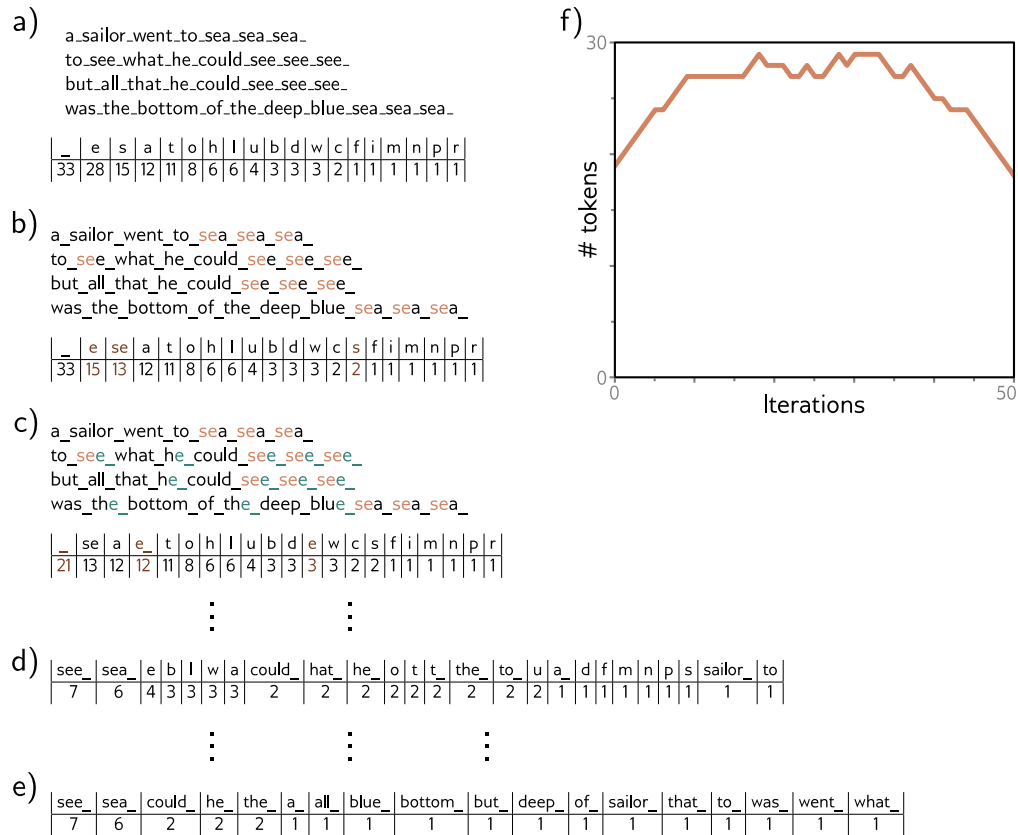


Figure 12.8 Sub-word tokenization. a) A passage of text from a nursery rhyme. The tokens are initially just the characters and whitespace (represented by an underscore), and their frequencies are displayed in the table. b) At each iteration, the sub-word tokenizer looks for the most commonly occurring adjacent pair of tokens (in this case, *se*) and merges them. This creates a new token and decreases the counts for the original tokens *s* and *e*. Note that the last character of the first token to be merged cannot be whitespace, which prevents merging across words. c) At the second iteration, the algorithm merges *e* and the whitespace character *_*. d) After 22 iterations, the tokens consist of a mix of letters, word fragments, and commonly occurring words. e) If we continue this process indefinitely, the tokens eventually represent the full words. f) Over time, the number of tokens increases as we add word fragments to the letters and then decreases again as we merge these fragments. In a real situation, there would be a very large number of words, and the algorithm would terminate when the vocabulary size (number of tokens) reached a predetermined value. Punctuation and capital letters would also be treated as separate input characters.

is mapped to a learned embedding. These embeddings are passed through a series of transformer layers. We now consider each of these stages in turn.

12.5.1 Tokenization

A text processing pipeline begins with a *tokenizer*. This splits the text into smaller constituent units (tokens) from a *vocabulary* of possible tokens. In the discussion above, we have implied that these tokens represent words, but there are several difficulties.

- Inevitably, some words (e.g., names) will not be in the vocabulary.
- It's unclear how to handle punctuation, but this is important. If a sentence ends in a question mark, we must encode this information.
- The vocabulary would need different tokens for versions of the same word with different suffixes (e.g., walk, walks, walked, walking), and there is no way to clarify that these variations are related.

One approach would be to use letters and punctuation marks as the vocabulary, but this would mean splitting text into very small parts and requiring the subsequent network to re-learn the relations between them.

In practice, a compromise between letters and full words is used, and the final vocabulary includes both common words and word fragments from which larger and less frequent words can be composed. The vocabulary is computed using a *sub-word tokenizer* such as *byte pair encoding* (figure 12.8) that greedily merges commonly occurring sub-strings based on their frequency.

Notebook 12.3
Tokenization

12.5.2 Embeddings

Each token in the vocabulary $\mathcal{V}$ is mapped to a unique *word embedding*, and the embeddings for the whole vocabulary are stored in a matrix $\mathbf{\Omega}_e \in \mathbb{R}^{D \times |\mathcal{V}|}$. To accomplish this, the N input tokens are first encoded in the matrix $\mathbf{T} \in \mathbb{R}^{|\mathcal{V}| \times N}$, where the n^{th} column corresponds to the n^{th} token and is a $|\mathcal{V}| \times 1$ *one-hot vector* (i.e., a vector where every entry is zero except for the entry corresponding to the token, which is set to one). The input embeddings are computed as $\mathbf{X} = \mathbf{\Omega}_e \mathbf{T}$, and $\mathbf{\Omega}_e$ is learned like any other network parameter (figure 12.9). A typical embedding size D is 1024, and a typical total vocabulary size $|\mathcal{V}|$ is 30,000, so even before the main network, there are many parameters in $\mathbf{\Omega}_e$ to learn.

12.5.3 Transformer model

Finally, the embedding matrix $\mathbf{X}$ representing the text is passed through a series of K transformer layers, called a *transformer model*. There are three types of transformer models. An *encoder* transforms the text embeddings into a representation that can support a variety of tasks. A *decoder* predicts the next token to continue the input

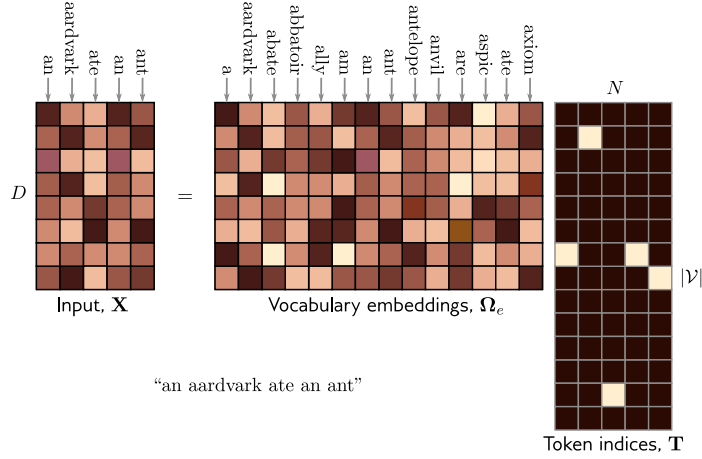


Figure 12.9 The input embedding matrix $\mathbf{X} \in \mathbb{R}^{D \times N}$ contains N embeddings of length D and is created by multiplying a matrix $\mathbf{\Omega}_e$ containing the embeddings for the entire vocabulary with a matrix containing one-hot vectors in its columns that correspond to the word or sub-word indices. The vocabulary matrix $\mathbf{\Omega}_e$ is considered a parameter of the model and is learned along with the other parameters. Note that the two embeddings for the word `an` in $\mathbf{X}$ are the same.

text. *Encoder-decoders* are used in *sequence-to-sequence tasks*, where one text string is converted into another (e.g., machine translation). These variations are described in sections 12.6–12.8, respectively.

12.6 Encoder model example: BERT

BERT is an encoder model that uses a vocabulary of 30,000 tokens. Input tokens are converted to 1024-dimensional word embeddings and passed through 24 transformer layers. Each contains a self-attention mechanism with 16 heads. The queries, keys, and values for each head are of dimension 64 (i.e., the matrices $\mathbf{\Omega}_{vh}, \mathbf{\Omega}_{qh}, \mathbf{\Omega}_{kh}$ are 64×1024). The dimension of the single hidden layer in the fully connected networks is 4096. The total number of parameters is ~ 340 million. When BERT was introduced, this was considered large, but it is now much smaller than state-of-the-art models.

Encoder models like BERT exploit *transfer learning* (section 9.3.6). During *pre-training*, the parameters of the transformer architecture are learned using *self-supervision* from a large corpus of text. The goal here is for the model to learn general information about the statistics of language. In the *fine-tuning stage*, the resulting network is adapted to solve a particular task using a smaller body of labelled training data.

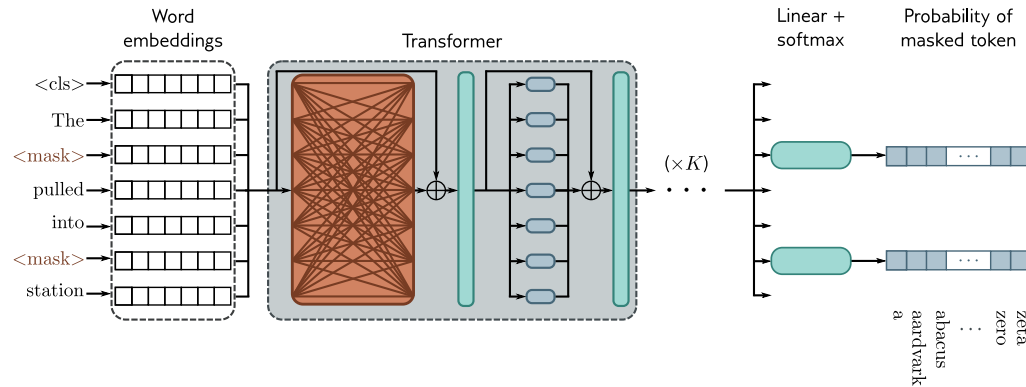


Figure 12.10 Pre-training for BERT-like encoder. The input tokens (and a special `<cls>` token denoting the start of the sequence) are converted to word embeddings. Here, these are represented as rows rather than columns, so the box labeled “word embeddings” is $\mathbf{X}^T$. These embeddings are passed through a series of transformer layers (orange connections indicate that every token attends to every other token in these layers) to create a set of output embeddings. A small fraction of the input tokens are randomly replaced with a generic `<mask>` token. In pre-training, the goal is to predict the missing word from the associated output embedding. To this end, the outputs corresponding to the masked tokens are passed through softmax functions, and a multiclass classification loss (section 5.24) is applied to each. This task has the advantage that it uses both the left and right context to predict the missing word but has the disadvantage that it does not make efficient use of data; here, seven tokens need to be processed to add two terms to the loss function.

12.6.1 Pre-training

In the pre-training stage, the network is trained using self-supervision. This allows the use of enormous amounts of data without the need for manual labels. For BERT, the self-supervision task consists of predicting missing words from sentences from a large internet corpus (figure 12.10).¹ During training, the maximum input length is 512 tokens, and the batch size is 256. The system is trained for a million steps, corresponding to roughly 50 epochs of the 3.3-billion word corpus.

Predicting missing words forces the transformer network to understand some syntax. For example, it might learn that the adjective `red` is often found before nouns like `house` or `car` but never before a verb like `shout`. It also allows the model to learn superficial *common sense* about the world. For example, after training, the model will assign a higher probability to the missing word `train` in the sentence `The <mask> pulled into the station` than it would to the word `peanut`. However, the degree of “understanding” this type of model can ever have is limited.

¹BERT also uses a secondary task that predicts whether two sentences were originally adjacent in the text or not, but this only marginally improves performance.

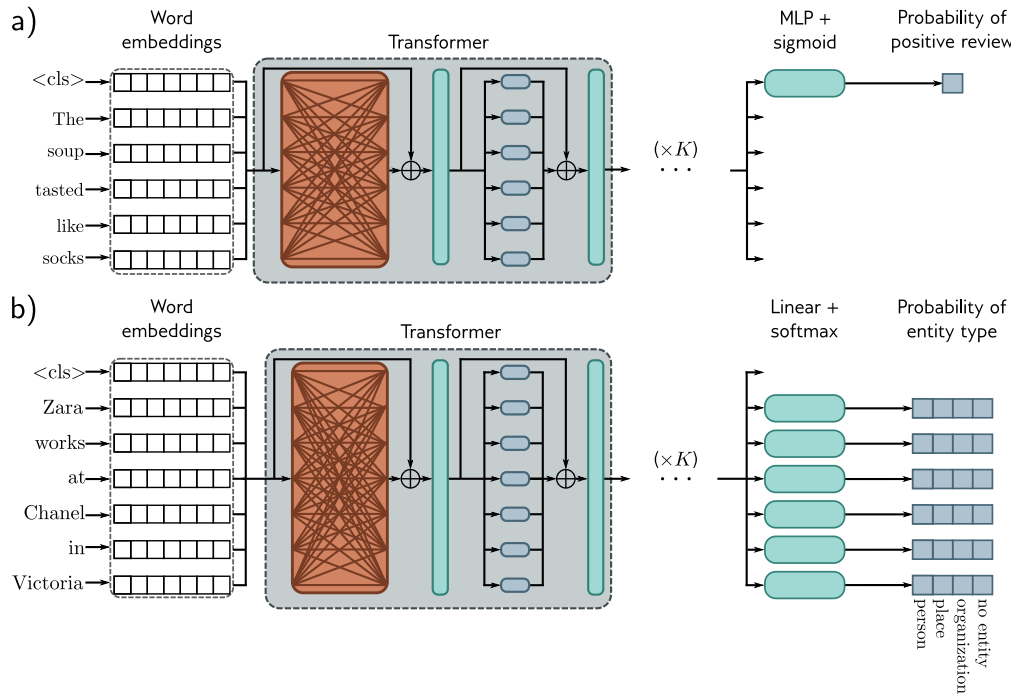


Figure 12.11 After pre-training, the encoder is fine-tuned using manually labeled data to solve a particular task. Usually, a linear transformation or a multi-layer perceptron (MLP) is appended to the encoder to produce whatever output is required. a) Example text classification task. In this sentiment classification task, the <cls> token embedding is used to predict the probability that the review is positive. b) Example word classification task. In this named entity recognition problem, the embedding for each word is used to predict whether the word corresponds to a person, place, or organization, or is not an entity.

12.6.2 Fine-tuning

In the fine-tuning stage, the model parameters are adjusted to specialize the network to a particular task. An extra layer is appended onto the transformer network to convert the output vectors to the desired output format. Examples include:

Text classification: In BERT, a special token known as the classification or <cls> token is placed at the start of each string during pre-training. For text classification tasks like *sentiment analysis* (in which the passage is labeled as having a positive or negative emotional tone), the vector associated with the <cls> token is mapped to a single number and passed through a logistic sigmoid (figure 12.11a). This contributes to a standard binary cross-entropy loss (section 5.4).

Word classification: The goal of *named entity recognition* is to classify each word as an entity type (e.g., person, place, organization, or no-entity). To this end, each input embedding $\mathbf{x}_n$ is mapped to an $E \times 1$ vector where the E entries correspond to the E entity types. This is passed through a softmax function to create probabilities for each class, which contribute to a multiclass cross-entropy loss (figure 12.11b).

Text span prediction: In the SQuAD 1.1 question answering task, the question and a passage from Wikipedia containing the answer are concatenated and tokenized. BERT is then used to predict the text span in the passage that contains the answer. Each token maps to two numbers indicating how likely it is that the text span begins and ends at this location. The resulting two sets of numbers are put through two softmax functions. The likelihood of any text span being the answer can be derived by combining the probability of starting and ending at the appropriate places.

12.7 Decoder model example: GPT3

This section presents a high-level description of GPT3, an example of a decoder model. The basic architecture is extremely similar to the encoder model and comprises a series of transformer layers that operate on learned word embeddings. However, the goal is different. The encoder aimed to build a representation of the text that could be fine-tuned to solve a variety of more specific NLP tasks. Conversely, the decoder has one purpose: to generate the next token in a sequence. It can generate a coherent text passage by feeding the extended sequence back into the model.

12.7.1 Language modeling

GPT3 is an autoregressive language model. This is easiest to understand with a concrete example. Consider the sentence *It takes great courage to let yourself appear weak*. For simplicity, let's assume that the tokens are the full words. The probability of the full sentence can be factored as:

$$\begin{aligned}
 Pr(\text{It takes great courage to let yourself appear weak}) &= \\
 &Pr(\text{It}) \times Pr(\text{takes}|\text{It}) \times Pr(\text{great}|\text{It takes}) \times Pr(\text{courage}|\text{It takes great}) \times \\
 &Pr(\text{to}|\text{It takes great courage}) \times Pr(\text{let}|\text{It takes great courage to}) \times \\
 &Pr(\text{yourself}|\text{It takes great courage to let}) \times \\
 &Pr(\text{appear}|\text{It takes great courage to let yourself}) \times \\
 &Pr(\text{weak}|\text{It takes great courage to let yourself appear}). \tag{12.14}
 \end{aligned}$$

An autoregressive model predicts the conditional distributions $Pr(t_n|t_1, \dots, t_{n-1})$ of each token given all the prior tokens, and hence indirectly computes the joint probability $Pr(t_1, t_2, \dots, t_N)$ of all N tokens:

$$Pr(t_1, t_2, \dots, t_N) = Pr(t_1) \prod_{n=2}^N Pr(t_n | t_1, \dots, t_{n-1}). \quad (12.15)$$

The autoregressive formulation demonstrates the connection between maximizing the joint probability of the tokens and the next token prediction task.

12.7.2 Masked self-attention

To train a decoder, we seek parameters that maximize the log probability of the input text under the autoregressive model (i.e., that maximize the sum of the log conditional probability terms). Ideally, we would pass in the whole sentence and compute all the log probabilities and gradients in the same forward pass rather than doing a forward pass for each token in the sentence. However, if we pass in the full sentence, the term computing $\log [Pr(\text{great} | \text{It takes})]$ would have access to both the answer **great** and the right context **courage to let yourself appear weak**. Hence, the system can cheat rather than learn to predict the following words and won't train properly.

Fortunately, the tokens only interact in the self-attention layers in a transformer network. Hence, the problem can be resolved by ensuring that the attention to the answer and the right context is zero. This can be achieved by setting the corresponding dot products in the self-attention computation (equation 12.5) to negative infinity before they are passed through the **softmax**[•] function. This is known as *masked self-attention*. The effect is to make the weight of all the upward-angled arrows in figure 12.1 zero.

The entire decoder network operates as follows. The input text is tokenized, and the tokens are converted to embeddings. The embeddings are passed into the transformer network, but now the transformer layers use masked self-attention so that they can only attend to the current and previous tokens. Each of the output embeddings can be thought of as representing a partial sentence, and for each, the goal is to predict the next token in the sequence. Consequently, after the transformer layers, a single linear layer maps each output embedding to the size of the vocabulary, followed by a **softmax**[•] function that converts these values to probabilities. During training, we aim to maximize the sum of the log probabilities of the next token in the ground truth sequence at every position using a standard multiclass cross-entropy loss (figure 12.12).

12.7.3 Generating text from a decoder

The autoregressive language model is the first example of a *generative model* discussed in this book. Since it defines a probability model over text sequences, it can be used to sample new examples of plausible text. To generate from the model, we start with an input sequence of text (which might be just the special **<start>** token indicating the beginning of the sequence) and feed this into the network, which then outputs the probabilities over possible subsequent tokens. We can then either pick the most likely token or sample from this probability distribution. The new extended sequence can be fed back into the decoder network to yield the probability distribution over the next

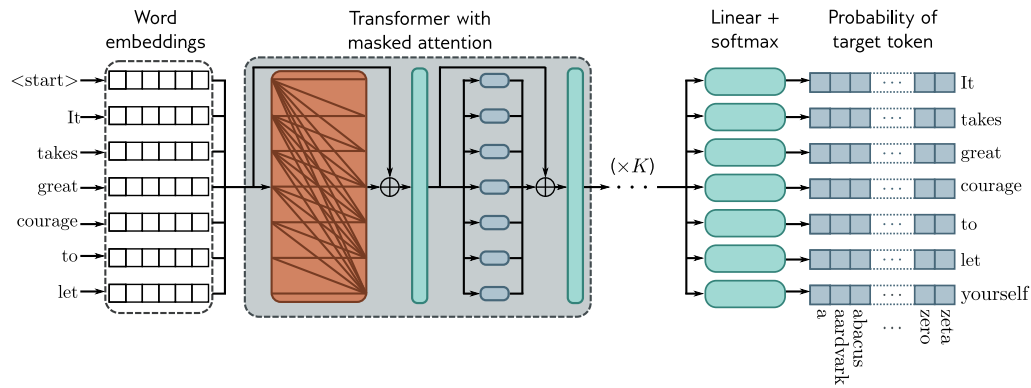


Figure 12.12 Training GPT3-type decoder network. The tokens are mapped to word embeddings with a special <start> token at the beginning of the sequence. The embeddings are passed through a series of transformer layers that use masked self-attention. Here, each position in the sentence can only attend to its own embedding and those of tokens earlier in the sequence (orange connections). The goal at each position is to maximize the probability of the following ground truth token in the sequence. In other words, at position one, we want to maximize the probability of the token *It*; at position two, we want to maximize the probability of the token *takes*; and so on. Masked self-attention ensures the system cannot cheat by looking at subsequent inputs. The autoregressive task has the advantage of making efficient use of the data since every word contributes a term to the loss function. However, it only exploits the left context of each word.

token. By repeating this process, we can generate large bodies of text. The computation can be made quite efficient as prior embeddings do not depend on subsequent ones due to the masked self-attention. Hence, much of the earlier computation can be recycled as we generate subsequent tokens.

In practice, many strategies can make the output text more coherent. For example, *beam search* keeps track of multiple possible sentence completions to find the overall most likely sequence of words (which is not necessarily found by greedily choosing the most likely word at each step). *Top-k sampling* randomly draws the next word from only the top-K most likely possibilities to prevent the system from accidentally choosing from the long tail of low-probability tokens and leading to an unnecessary linguistic dead end.

12.7.4 GPT3 and few-shot learning

Large language models like GPT3 apply these ideas on a massive scale. In GPT3, the sequence lengths are 2048 tokens long, and the total batch size is 3.2 million tokens. There are 96 transformer layers (some of which implement a sparse version of attention), each processing a word embedding of size 12288. There are 96 heads in the self-attention

layers, and the value, query, and key dimension is 128. It is trained with 300 billion tokens and contains 175 billion parameters.

Here's an example of completing text from the GPT3 model where the text provided to the model is in orange, and the generated text is in cyan:

Understanding Deep Learning is a new textbook from MIT Press by Simon Prince that's designed to offer an accessible, broad introduction to the field. Deep learning is a branch of machine learning that is concerned with algorithms that learn from data that is unstructured or unlabeled. The book is divided into four sections:

1. Introduction to deep learning
2. Deep learning architecture
3. Deep learning algorithms
4. Applications of deep learning

The first section offers an introduction to deep learning, including its history and origins. The second section covers deep learning architecture, discussing various types of neural networks and their applications. The third section dives into deep learning algorithms, including supervised and unsupervised learning, reinforcement learning, and more. The fourth section applies deep learning to various domains, such as computer vision, natural language processing, and robotics.

This continuation is plausible, albeit only partially accurate.

One surprising property of learning models on this scale is that they can perform many tasks without fine-tuning. If we provide several examples of correct question/answer pairs and then another question, they often answer the final question correctly by completing the sequence. One example is correcting English grammar:

Poor English input: I eated the purple berries.

Good English output: I ate the purple berries.

Poor English input: Thank you for picking me as your designer. I'd appreciate it.

Good English output: Thank you for choosing me as your designer. I appreciate it.

Poor English input: The mentioned changes have done. or I did the alteration that you requested. or I changed things you wanted and did the modifications.

Good English output: The requested changes have been made. or I made the alteration that you requested. or I changed things you wanted and made the modifications.

Poor English input: I'd be more than happy to work with you in another project.

Good English output: I'd be more than happy to work with you on another project.

(result from Brown et al., 2020)

Here, the text containing the paired examples in orange was provided as context for GPT3, and the system then generated the correct answer in cyan. This phenomenon extends to many situations, including generating code snippets based on natural language descriptions, arithmetic, translating between languages, and answering questions about text passages. Consequently, it is argued that enormous language models are *few-shot learners*; they can learn to do novel tasks based on just a few examples. However, performance is erratic in practice, and the extent to which it is extrapolating from learned examples rather than merely interpolating or copying verbatim is unclear.

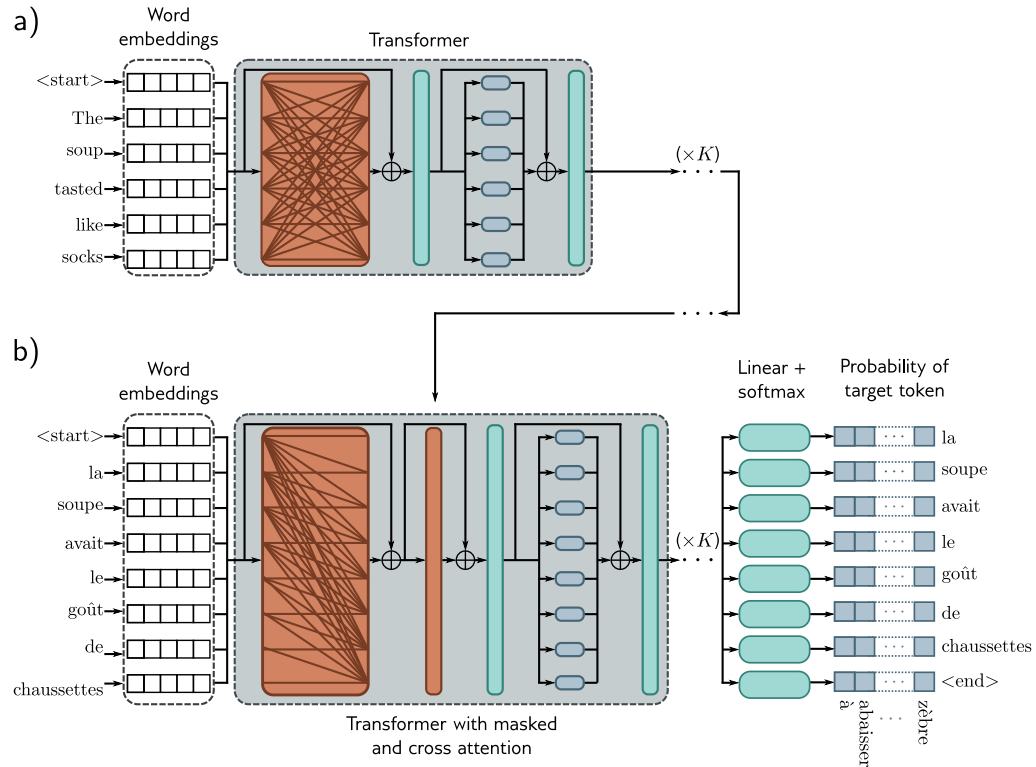


Figure 12.13 Encoder-decoder architecture. Two sentences are passed to the system with the goal of translating the first into the second. a) The first sentence is passed through a standard encoder. b) The second sentence is passed through a decoder that uses masked self-attention but also attends to the output embeddings of the encoder using cross-attention (orange rectangle). The loss function is the same as for the decoder model; we want to maximize the probability of the next word in the output sequence.

12.8 Encoder-decoder model example: machine translation

Translation between languages is an example of a *sequence-to-sequence* task. One common approach uses both an encoder (to compute a good representation of the source sentence) and a decoder (to generate the sentence in the target language). This is aptly called an *encoder-decoder* model.

Consider translating from English to French. The encoder receives the sentence in English and processes it through a series of transformer layers to create an output representation for each token. During training, the decoder receives the ground truth translation in French and passes it through a series of transformer layers that use masked self-attention and predict the following word at each position. However, the decoder layers also attend to the output of the encoder. Consequently, each French output word is

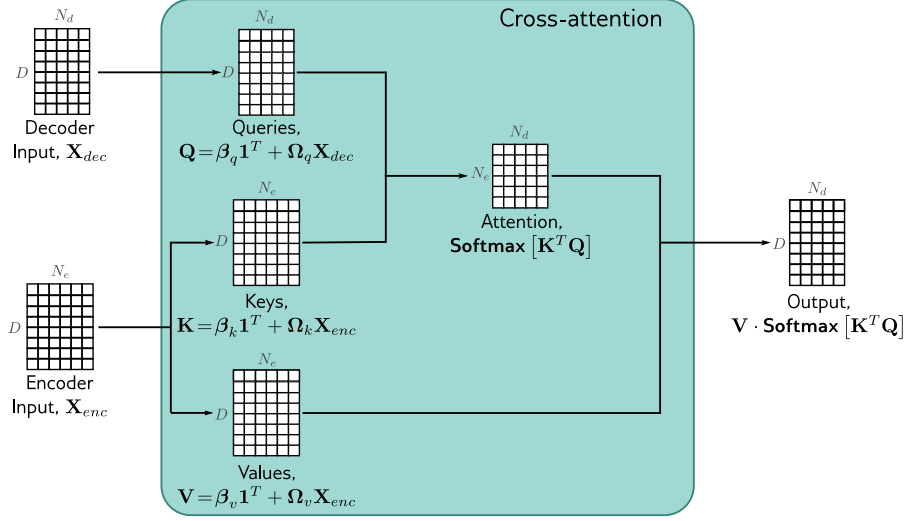


Figure 12.14 Cross-attention. The flow of computation is the same as in standard self-attention, but the queries are calculated from the decoder embeddings $\mathbf{X}_{dec}$, and the keys and values from the encoder embeddings $\mathbf{X}_{enc}$. For translation tasks, the encoder contains information about the source language statistics, and the decoder contains information about the target language statistics.

conditioned on the previous output words *and* the source English sentence (figure 12.13).

This is achieved by modifying the transformer layers in the decoder. Originally, these consisted of a masked self-attention layer followed by a neural network applied individually to each embedding (figure 12.12). A new self-attention layer is added between these two components, in which the decoder embeddings attend to the encoder embeddings. This uses a version of self-attention known as *encoder-decoder attention* or *cross-attention*, where the queries are computed from the decoder embeddings and the keys and values from the encoder embeddings (figure 12.14).

12.9 Transformers for long sequences

Since each token in a transformer encoder model interacts with every other token, the computational complexity scales quadratically with the length of the sequence. For a decoder model, each token only interacts with previous tokens, so there are roughly half the number of interactions, but the complexity still scales quadratically. These relationships can be visualized as interaction matrices (figure 12.15a–b).

This quadratic increase in the amount of computation ultimately limits the length of sequences that can be used. Many methods have been developed to extend the trans-

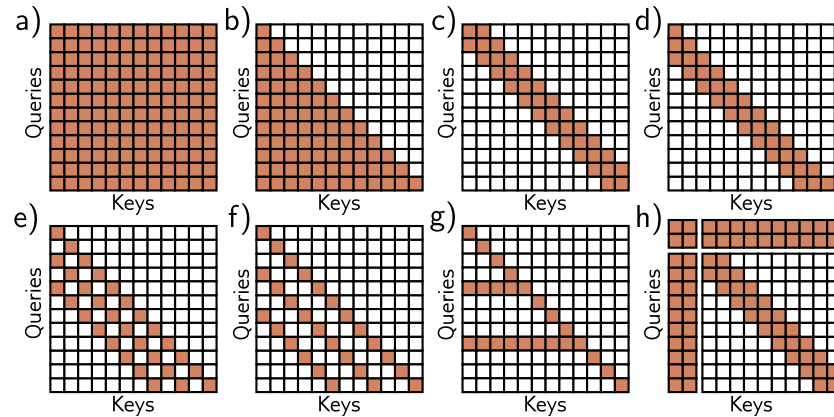


Figure 12.15 Interaction matrices for self-attention. a) In an encoder, every token interacts with every other token, and computation expands quadratically with the number of tokens. b) In a decoder, each token only interacts with the previous tokens, but complexity is still quadratic. c) Complexity can be reduced by using a convolutional structure (encoder case). d) Convolutional structure for decoder case. e–f) Convolutional structure with dilation rate of two and three (decoder case). g) Another strategy is to allow selected tokens to interact with all the other tokens (encoder case) or all the previous tokens (decoder case pictured). h) Alternatively, global tokens can be introduced (left two columns and top two rows). These interact with all of the tokens as well as with each other.

former to cope with longer sequences. One approach is to prune the self-attention interactions or, equivalently, to sparsify the interaction matrix (figures 12.15c–h). For example, this can be restricted to a convolutional structure so that each token only interacts with a few neighboring tokens. Across multiple layers, tokens still interact at larger distances as the receptive field expands. As for convolution in images, the kernel can vary in size and dilation rate.

A pure convolutional approach requires many layers to integrate information over large distances. One way to speed up this process is to allow select tokens (perhaps at the start of every sentence) to attend to all other tokens (encoder model) or all previous tokens (decoder model). A similar idea is to have a small number of global tokens that connect to all the other tokens and themselves. Like the `<cls>` token, these do not represent any word but serve to provide long-distance connections.

12.10 Transformers for images

Transformers were initially developed for text data. Their enormous success in this area led to experimentation on images. This was not obviously a promising idea for two

reasons. First, there are many more pixels in an image than words in a sentence, so the quadratic complexity of self-attention poses a practical bottleneck. Second, convolutional nets have a good inductive bias because each layer is equivariant to spatial translation, and takes into account the 2D structure of the image. However, this must be learned in a transformer network.

Regardless of these apparent disadvantages, transformer networks for images have now eclipsed the performance of convolutional networks for image classification and other tasks. This is partly because of the enormous scale at which they can be constructed and the large amounts of data that can be used to pre-train the networks. This section describes transformer models for images.

12.10.1 ImageGPT

ImageGPT is a transformer decoder; it builds an autoregressive model of image pixels that ingests a partial image and predicts the subsequent pixel value. The quadratic complexity of the transformer network means that the largest model (which contained 6.8 billion parameters) could still only operate on 64×64 images. Moreover, to make this tractable, the original 24-bit RGB color space had to be quantized into a nine-bit color space, so the system ingests (and predicts) one of 512 possible tokens at each position.

Images are naturally 2D objects, but ImageGPT simply learns a different positional encoding at each pixel. Hence it must learn that each pixel has a close relationship with its preceding neighbors and also with nearby pixels in the row above. Figure 12.16 shows example generation results.

The internal representation of this decoder was used as a basis for image classification. Each pixel's final embedding is averaged, and a linear layer maps these values to activations which are passed through a softmax layer to predict class probabilities. The system is pre-trained on a large corpus of web images and then fine-tuned on the ImageNet database resized to 48×48 pixels using a loss function that contains both a cross-entropy term for image classification and a generative loss term for predicting the pixels. Despite using a large amount of external training data, the system achieved only a 27.4% top-1 error rate on ImageNet (figure 10.15). This was worse than convolutional architectures of the time (see figure 10.21) but is still impressive given the small input image size; unsurprisingly, it fails where the target object is small or thin.

12.10.2 Vision Transformer (ViT)

The *Vision Transformer* tackled the problem of image resolution by dividing the image into 16×16 patches (figure 12.17). Each patch is mapped to an input embedding via a learned linear transformation, and these representations are fed into the transformer network. Once again, standard 1D positional encodings are learned.

Problem 12.8

This is an encoder model with a `<cls>` token (see figures 12.10–12.11). However, unlike BERT, it uses *supervised* pre-training on a large database of 303 million labeled images from 18,000 classes. The `<cls>` token is mapped via a final network layer to create activations that are fed into a softmax function to generate class probabilities. After pre-training, the system is applied to the final classification task by replacing this

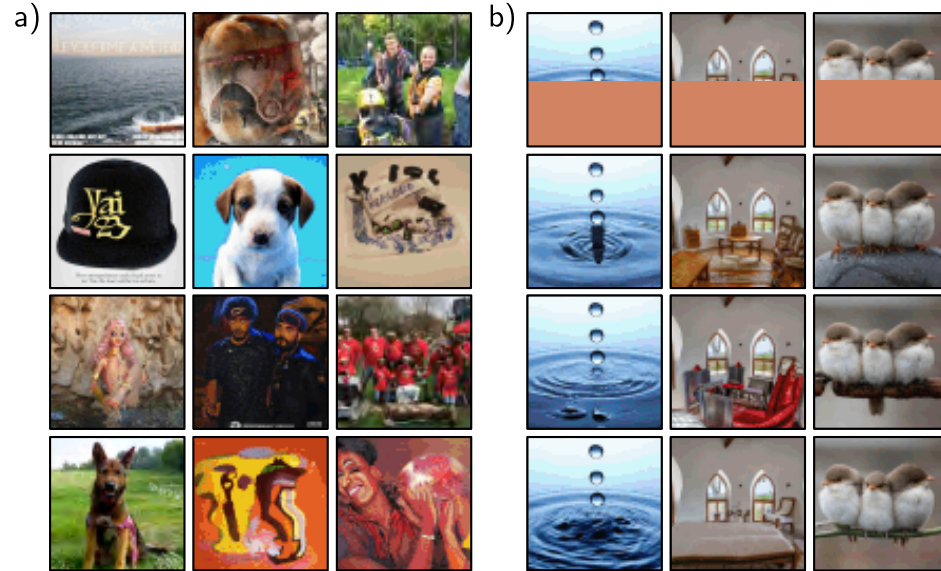


Figure 12.16 ImageGPT. a) Images generated from the autoregressive ImageGPT model. The top-left pixel is drawn from the estimated empirical distribution at this position. Subsequent pixels are generated in turn, conditioned on the previous ones, working along the rows until the bottom-right of the image is reached. For each pixel, the transformer decoder generates a conditional distribution as in equation 12.15, and a sample is drawn. The extended sequence is then fed back into the network to generate the next pixel, and so on. b) Image completion. In each case, the lower half of the image is removed (top row), and ImageGPT completes the remaining part pixel by pixel (three different completions shown). Adapted from <https://openai.com/blog/image-gpt/>.

final layer with one that maps to the desired number of classes and is fine-tuned.

For the ImageNet benchmark, this system achieved an 11.45% top-1 error rate. However, it did not perform as well as the best contemporary convolutional networks without supervised pre-training. The strong inductive bias of convolutional networks can only be superseded by employing extremely large amounts of training data.

12.10.3 Multi-scale vision transformers

The Vision Transformer differs from convolutional architectures in that it operates on a single scale and has a receptive field that covers the whole image. Several transformer models that process the image at multiple scales have been proposed. Similarly to convolutional networks, these generally start with small high resolution patches and few channels and gradually enlarge the receptive field, decrease the spatial resolution and

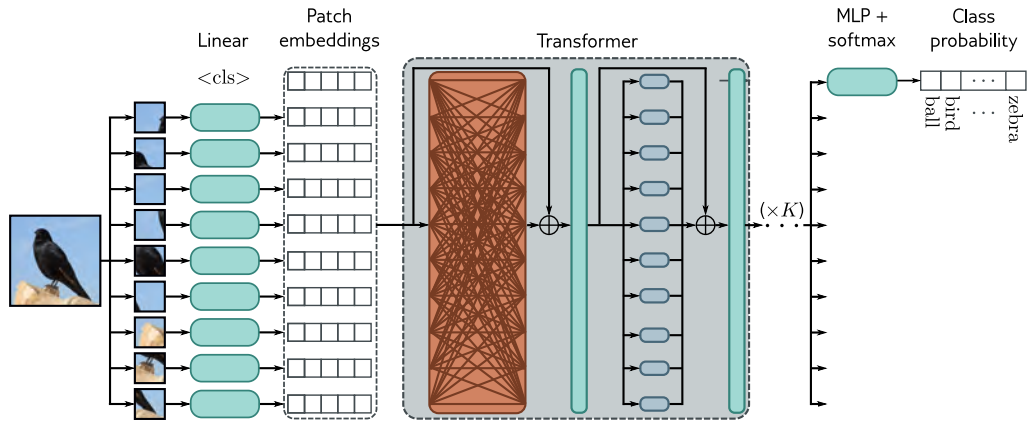


Figure 12.17 Vision transformer. The Vision Transformer (ViT) breaks the image into a grid of patches (16×16 in the original implementation). Each of these is projected via a learned linear transformation to become a patch embedding. These patch embeddings are fed into a transformer encoder network, and the $\langle \text{cls} \rangle$ token is used to predict the class probabilities.

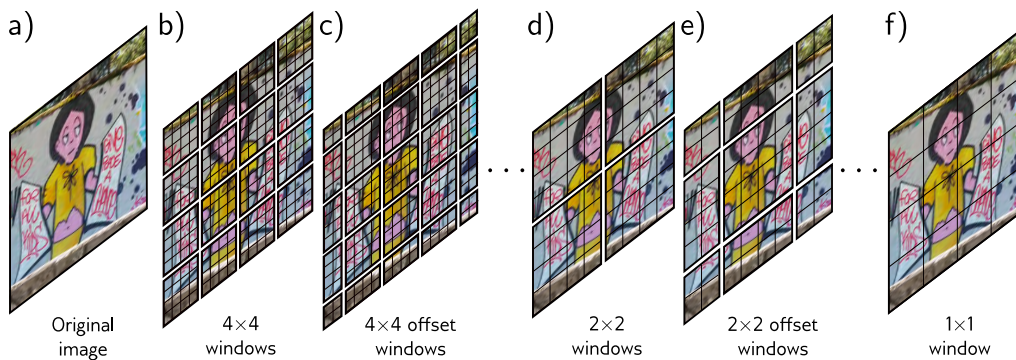


Figure 12.18 Shifted window (SWin) transformer (Liu et al., 2021c). a) Original image. b) The SWin transformer breaks the image into a grid of windows and each of these windows into a sub-grid of patches. The transformer network applies self-attention to the patches within each window independently (i.e., patches only attend to other patches in the same window). c) Each alternate layer shifts the windows so that the subsets of patches that interact with one another change, and information can propagate across the whole image. d) After several layers, the 2×2 blocks of patch representations are concatenated to increase the effective patch (and window) size. e) Alternate layers use shifted windows at this new lower resolution. f) Eventually, the resolution is such that there is just a single window, and the patches span the entire image.

increase the number of channels (embedding dimension).

A representative example of a multi-scale transformer is the *shifted-window* or *SWin* transformer. This is an encoder transformer that divides the image into patches and groups these patches into a grid of windows within which self-attention is applied independently (figure 12.18). These windows are shifted in adjacent transformers, so the effective receptive field at a given patch can expand beyond the window border.

The scale is reduced periodically by concatenating features from non-overlapping 2×2 patches and applying a linear transformation that maps these concatenated features to twice the original number of channels. This architecture does not have a `<cls>` token but instead averages the output features at the last layer. These are then mapped via a linear layer to the desired number of classes and passed through a softmax function to output class probabilities. At the time of writing, the most sophisticated version of this architecture achieves a 9.89% top-1 error rate on the ImageNet database.

A related idea is periodically to integrate information from across the whole image. *Dual attention vision transformers* (DaViT) alternate two types of transformers. In the first, image patches attend to one another, and the self-attention computation uses all the channels. In the second, the channels attend to one another, and the self-attention computation uses all the image patches. This architecture reaches a 9.60% top-1 error rate on ImageNet and is close to the state-of-the-art at the time of writing.

Problem 12.9

12.11 Summary

This chapter introduced self-attention and the transformer architecture. Encoder, decoder, and encoder-decoder models were then described. The transformer operates on sets of high-dimensional embeddings. It has a low computational complexity per layer, and much of the computation can be performed in parallel using the matrix form. Since every input embedding interacts with every other, it can describe long-range dependencies in text. Ultimately, the computation scales quadratically with the sequence length; one approach to reducing the complexity is sparsifying the interaction matrix.

The training of transformers with very large unlabeled datasets is the first example of *unsupervised learning* (learning without labels) in this book. Encoders learn a representation that can be used for other tasks by predicting missing tokens. Decoders build an autoregressive model over the inputs and are the first example of a *generative model* in this book. The generative decoders can be used to create new data examples.

Chapter 13 considers networks for processing graph data. These have connections with transformers in that the nodes of the graph attend to one another in each network layer. Chapters 14–18 return to unsupervised learning and generative models.

Notes

Natural language processing: Transformers were developed for natural language processing (NLP) tasks. This is an enormous area that deals with text analysis, categorization, generation, and manipulation. Example tasks include part of speech tagging, translation, text classification,

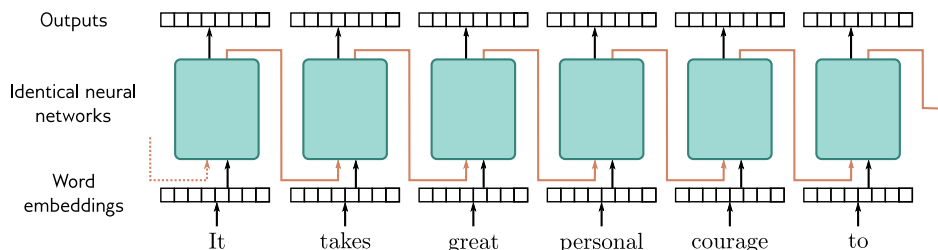


Figure 12.19 Recurrent neural networks (RNNs). The word embeddings are passed sequentially through a series of identical neural networks. Each network has two outputs; one is the output embedding, and the other (orange arrows) feeds back into the next neural network, along with the next word embedding. Each output embedding contains information about the word itself and its context in the preceding sentence fragment. In principle, the final output contains information about the entire sentence and could be used to support classification tasks similarly to the `<cls>` token in a transformer encoder model. However, RNNs sometimes gradually “forget” about tokens that are further back in time.

entity recognition (people, places, companies, etc.), text summarization, question answering, word sense disambiguation, and document clustering. NLP was initially tackled by rule-based methods that exploited the structure and statistics of grammar. See Manning & Schütze (1999) and Jurafsky & Martin (2000) for early approaches.

Recurrent neural networks: Before the introduction of transformers, many state-of-the-art NLP applications used *recurrent neural networks*, or *RNNs* for short (figure 12.19). The term “recurrent” was introduced by Rumelhart et al. (1985), but the main idea dates to at least Minsky & Papert (1969). RNNs ingest a sequence of inputs (words in NLP) one at a time. At each step, the network receives both the new input and a hidden representation computed from the previous time step (the recurrent connection). The final output contains information about the whole input. This representation can then support NLP tasks like classification or translation. They have also been used in a decoding context in which generated tokens are fed back into the model to form the next input to the sequence. For example, the PixelRNN (Van den Oord et al., 2016c) used RNNs to build an autoregressive model of images.

From RNNs to transformers: One of the problems with RNNs is that they can forget information that is further back in the sequence. More sophisticated versions of this architecture, such as *long short-term memory networks* or *LSTMs* (Hochreiter & Schmidhuber, 1997b) and *gated recurrent units* or *GRUs* (Cho et al., 2014; Chung et al., 2014) partially addressed this problem. However, in machine translation, the idea emerged that all of the intermediate representations in the RNN could be exploited to produce the output sentence. Moreover, certain output words should *attend* more to certain input words according to their relation (Bahdanau et al., 2015). This ultimately led to dispensing with the recurrent structure and replacing it with the encoder-decoder transformer (Vaswani et al., 2017). Here input tokens attend to one another (self-attention), output tokens attend to those earlier in the sequence (masked self-attention), and output tokens also attend to the input tokens (cross-attention). A formal algorithmic description of the transformer can be found in Phuong & Hutter (2022), and a survey of work can be found in Lin et al. (2022). The literature should be approached with caution, as many en-

hancements to transformers do not make meaningful performance improvements when carefully assessed in controlled experiments (Narang et al., 2021).

Applications: Models based on self-attention and/or the transformer architecture have been applied to text sequences (Vaswani et al., 2017), image patches (Dosovitskiy et al., 2021), protein sequences (Rives et al., 2021), graphs (Veličković et al., 2019), database schema (Xu et al., 2021b), speech (Wang et al., 2020c), mathematical integration when formulated as a translation problem (Lample & Charton, 2020), and time series (Wu et al., 2020b). However, their most celebrated successes have been in building language models and, more recently, as a replacement for convolutional networks in computer vision.

Large language models: Vaswani et al. (2017) targeted translation tasks, but transformers are now more usually used to build either pure encoder or pure decoder models, the most famous of which are BERT (Devlin et al., 2019) and GPT2/GPT3 (Radford et al., 2019; Brown et al., 2020), respectively. These models are usually tested against benchmarks like GLUE (Wang et al., 2019b), which includes the SQuAD question-answering task (Rajpurkar et al., 2016) described in section 12.6.2, SuperGLUE (Wang et al., 2019a) and BIG-bench (Srivastava et al., 2022), which combine many NLP tasks to create an aggregate score for measuring language ability. Decoder models are generally not fine-tuned for these tasks but can perform well anyway when given a few examples of questions and answers and asked to complete the text from the next question. This is referred to as *few-shot learning* (Brown et al., 2020).

Since GPT3, many decoder language models have been released with steady improvement in few-shot results. These include GLaM (Du et al., 2022), Gopher (Rae et al., 2021), Chinchilla (Hoffmann et al., 2023), Megatron-Turing NLG (Smith et al., 2022), and LaMDa (Thoppilan et al., 2022). Most of the performance improvement is attributable to increased model size, using sparsely activated modules, and exploiting larger datasets. At the time of writing, the most recent model is PaLM (Chowdhery et al., 2022), which has 540 billion parameters and was trained on 780 billion tokens across 6144 processors. Interestingly, since text is highly compressible, this model has more than enough capacity to memorize the entire training dataset. This is true for many language models. Many bold statements have been made about how large language models exceed human performance. This is probably true for some tasks, but such statements should be treated with caution (see Ribeiro et al., 2021; McCoy et al., 2019; Bowman & Dahl, 2021; and Dehghani et al., 2021).

These models have considerable world knowledge. For example, in section 12.7.4, the model knows key facts about deep learning, including that it is a type of machine learning with associated algorithms and applications. Indeed, one such model has been mistakenly identified as being sentient (Clark, 2022). However, there are persuasive arguments that the degree of “understanding” this type of model can ever have is limited (Bender & Koller, 2020).

Tokenizers: Schuster & Nakajima (2012) and Sennrich et al. (2015) introduced *WordPiece* and *byte pair encoding (BPE)*, respectively. Both methods greedily merge pairs of tokens based on their frequency of adjacency (figure 12.8), with the main difference being how the initial tokens are chosen. For example, in BPE, the initial tokens are characters or punctuation with a special token to denote whitespace. The merges cannot occur over the whitespace. As the algorithm proceeds, new tokens are formed by combining characters recursively so that sub-word and word tokens emerge. The unigram language model (Kudo, 2018) generates several possible candidate merges and chooses the best one based on the likelihood in a language model. Provilkov et al. (2020) develop BPE dropout, which generates the candidates more efficiently by introducing randomness into the process of counting frequencies. Versions of both byte pair encoding and the unigram language model are included in the SentencePiece library (Kudo & Richardson, 2018), which works directly on Unicode characters and can work with any language. He et al. (2020) introduce a method that treats the sub-word segmentation as a latent variable that should be marginalized out for learning and inference.

Decoding algorithms: Transformer decoder models take a body of text and return a probability over the next token. This is then added to the preceding text, and the model is run again. The process of choosing tokens from these probability distributions is known as *decoding*. Naïve ways to do this would be to either (i) greedily choose the most likely token or (ii) choose a token randomly according to the distribution. However, neither of these methods works well in practice. In the former case, the results may be very generic, and the latter case may lead to degraded quality outputs (Holtzman et al., 2020). This is partly because, during training, the model was only exposed to sequences of ground truth tokens (known as *teacher forcing*) but sees its own output when deployed.

It is not computationally feasible to try every combination of tokens in the output sequence, but it is possible to maintain a fixed number of parallel hypotheses and choose the most likely overall sequence. This is known as *beam search*. Beam search tends to produce many similar hypotheses and has been modified to investigate more diverse sequences (Vijayakumar et al., 2016; Kulikov et al., 2018). One possible problem with random sampling is that there is a very long tail of unlikely following words that collectively have a significant probability. This has led to the development of *top-K sampling*, in which tokens are sampled from only the K most likely hypotheses (Fan et al., 2018). Top-K sampling still sometimes allows unreasonable token choices when there are only a few high-probability choices. To resolve this problem, Holtzman et al. (2020) proposed *nucleus sampling*, in which tokens are sampled from a fixed proportion of the total probability mass. El Asri & Prince (2020) discuss decoding algorithms in more depth.

Types of attention: Scaled dot-product attention (Vaswani et al., 2017) is just one of a family of attention mechanisms that includes additive attention (Bahdanau et al., 2015), multiplicative attention (Luong et al., 2015), key-value attention (Daniluk et al., 2017), and memory-compressed attention (Liu et al., 2019c). Zhai et al. (2021) constructed “attention-free” transformers, in which the tokens interact in a way that does not have quadratic complexity. Multi-head attention was also introduced by Vaswani et al. (2017). Interestingly, it appears that most of the heads can be pruned after training without critically affecting the performance (Voita et al., 2019); it has been suggested that their role is to guard against bad initializations. Hu et al. (2018b) propose squeeze-and-excitation networks, attention-like mechanisms that re-weight the channels in a convolutional layer based on globally computed features.

Relationship of self-attention to other models: The self-attention computation has close connections to other models. First, it is an example of a hypernetwork (Ha et al., 2017) in that it uses one part of the network to choose the weights of another part: the attention matrix forms the weights of a sparse network layer that maps the values to the outputs (figure 12.3). The *synthesizer* (Tay et al., 2021) simplifies this idea by using a neural network to create each row of the attention matrix from the corresponding input embedding. Even though the input tokens no longer interact with each other to create the attention weights, this works surprisingly well. Wu et al. (2019) present a similar system that produces an attention matrix with a convolutional structure so the tokens attend to their neighbors. The gated multi-layer perceptron (Wu et al., 2019) computes a matrix that pointwise multiplies the values and hence modifies them without mixing them. Transformers are also closely related to *fast weight memory systems*, which were the intellectual forerunners of hypernetworks (Schlag et al., 2021).

Self-attention can also be thought of as a routing mechanism (figure 12.1), and from this viewpoint, there is a connection to capsule networks (Sabour et al., 2017). These capture hierarchical relations in images; lower network levels might detect facial parts (noses, mouths), which are then combined (routed) in higher-level capsules that represent a face. However, capsule networks use *routing by agreement*. In self-attention, the inputs compete with each other for how much they contribute to a given output (via the softmax operation). In capsule networks, the outputs of the layer compete with each other for inputs from earlier layers. Once we consider self-attention as a routing network, we can question whether making this routing dynamic (i.e., dependent on the data) is necessary. The random synthesizer (Tay et al., 2021) removed the de-

pendence of the attention matrix on the inputs entirely and either used predetermined random values or learned values. This performed surprisingly well across a variety of tasks.

Multi-head self-attention also has close connections to graph neural networks (see chapter 13), convolution (Cordonnier et al., 2020), recurrent neural networks (Choromanski et al., 2020), and memory retrieval in Hopfield networks (Ramsauer et al., 2021). For more information on the relationships between transformers and other models, consult Prince (2021a).

Positional encoding: The original transformer paper (Vaswani et al., 2017) experimented with predefining the positional encoding matrix $\mathbf{\Pi}$, and learning the positional encoding $\mathbf{\Pi}$. It might seem odd to *add* the positional encodings to the $D \times N$ data matrix $\mathbf{X}$ rather than concatenate them. However, the data dimension D is usually greater than the number of tokens N , so the positional encoding lies in a subspace. The word embeddings in $\mathbf{X}$ are learned, so the system can theoretically keep the two components in orthogonal subspaces and retrieve the positional encodings as required. The predefined embeddings chosen by Vaswani et al. (2017) were a family of sinusoidal components with two attractive properties: (i) the relative position of two embeddings is easy to recover using a linear operation and (ii) their dot product generally decreased as the distance between positions increased (see Prince, 2021a, for more details). Many systems, such as GPT3 and BERT, learn positional encodings. Wang et al. (2020a) examined the cosine similarities of the positional encodings in these models and showed that they generally decline with relative distance, although they also have a periodic component.

Much subsequent work has modified just the attention matrix so that in the scaled dot-product self-attention equation:

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax} \left[\frac{\mathbf{K}^T \mathbf{Q}}{\sqrt{D_q}} \right], \quad (12.16)$$

only the queries and keys contain position information:

$$\begin{aligned} \mathbf{V} &= \beta_v \mathbf{1}^T + \Omega_v \mathbf{X} \\ \mathbf{Q} &= \beta_q \mathbf{1}^T + \Omega_q (\mathbf{X} + \mathbf{\Pi}) \\ \mathbf{K} &= \beta_k \mathbf{1}^T + \Omega_k (\mathbf{X} + \mathbf{\Pi}). \end{aligned} \quad (12.17)$$

This has led to the idea of multiplying out the quadratic component in the numerator of equation 12.16 and retaining only some of the terms. For example, Ke et al. (2021) decouple or *untie* the content and position information by retaining only the content-content and position-position terms and using different projection matrices $\Omega_\bullet$ for each.

Another modification is to inject information directly about the relative position. This is more important than absolute position since a batch of text can start at an arbitrary place in a document. Shaw et al. (2018), Raffel et al. (2020), and Huang et al. (2020b) all developed systems where a single term was learned for each relative position offset, and the attention matrix was modified in various ways using these *relative positional encodings*. Wei et al. (2019) investigated relative positional encodings based on predefined sinusoidal embeddings rather than learned values. DeBERTa (He et al., 2021) combines these ideas; they retain only a subset of terms from the quadratic expansion, apply different projection matrices to them, and use relative positional encodings. Other work has explored sinusoidal embeddings that encode absolute and relative position information in more complex ways (Su et al., 2021).

Wang et al. (2020a) compare the performance of transformers in BERT with different positional encodings. They found that relative positional encodings perform better than absolute positional encodings, but there was little difference between using sinusoidal and learned embeddings. A survey of positional encodings can be found in Dufter et al. (2021).

Extending transformers to longer sequences: The complexity of the self-attention mechanism increases quadratically with the sequence length. Some tasks like summarization or question answering may require long inputs, so this quadratic dependence limits performance. Three lines of work have attempted to address this problem. The first decreases the size of the attention matrix, the second makes the attention sparse, and the third modifies the attention mechanism to make it more efficient.

To decrease the size of the attention matrix, Liu et al. (2018b) introduced *memory-compressed attention*. This applies strided convolution to the keys and values, which reduces the number of positions in a very similar way to downsampling in a convolutional network. Attention is now applied between weighted combinations of neighboring positions, where the weights are learned. Along similar lines, Wang et al. (2020b) observed that the quantities in the attention mechanism are often low rank in practice and developed the *LinFormer*, which projects the keys and values onto a smaller subspace before computing the attention matrix.

To make attention sparse, Liu et al. (2018b) proposed *local attention*, in which neighboring blocks of tokens only attend to one another. This creates a block diagonal interaction matrix (see figure 12.15). Information cannot pass from block to block, so such layers are typically alternated with full attention. Along the same lines, GPT3 (Brown et al., 2020) uses a convolutional interaction matrix and alternates this with full attention. Child et al. (2019) and Beltagy et al. (2020) experimented with various interaction matrices, including convolutional structures with different dilation rates but allowing some queries to interact with every other key. Ainslie et al. (2020) introduced the *extended transformer construction* (figure 12.15h), which uses a set of global embeddings that interact with every other token. This can only be done in the encoder version, or these implicitly allow the system to “look ahead.” When combined with relative position encoding, this scheme requires special encodings for mapping to, from, and between these global embeddings. *BigBird* (Ainslie et al., 2020) combined global embeddings and a convolutional structure with a random sampling of possible connections. Other work has investigated learning the sparsity pattern of the attention matrix (Roy et al., 2021; Kitaev et al., 2020; Tay et al., 2020).

Finally, it has been noted that the terms in the numerator and denominator of the softmax operation that computes attention have the form $\exp[\mathbf{k}^T \mathbf{q}]$. This can be treated as a kernel function and, as such, can be expressed as the dot product $\mathbf{g}[\mathbf{k}]^T \mathbf{g}[\mathbf{q}]$ where $\mathbf{g}[\bullet]$ is a nonlinear transformation and, as such, can be expressed as the dot product $\mathbf{g}[\mathbf{k}]^T \mathbf{g}[\mathbf{q}]$ where $\mathbf{g}[\bullet]$ is a nonlinear transformation. This formulation decouples the queries and keys, making the attention computation more efficient. Unfortunately, to replicate the form of the exponential terms, the transformation $\mathbf{g}[\bullet]$ must map the inputs to the infinite space. The linear transformer (Katharopoulos et al., 2020) recognizes this and replaces the exponential term with a different similarity measure. The *Performer* (Choromanski et al., 2020) approximates this infinite mapping with a finite-dimensional one. More details about extending transformers to longer sequences can be found in Tay et al. (2023) and Prince (2021a).

Problem 12.10

Training transformers: Training transformers is challenging and requires both learning rate warm-up (Goyal et al., 2018) and Adam (Kingma & Ba, 2015). Indeed Xiong et al. (2020a) and Huang et al. (2020a) show that the gradients vanish, and the Adam updates decrease in magnitude without learning rate warm-up. Several interacting factors cause this problem. Residual connections cause the exploding gradients (figure 11.6), but normalization layers prevent this. Vaswani et al. (2017) used LayerNorm rather than BatchNorm because NLP statistics are highly variable between batches, although subsequent work has modified BatchNorm for transformers (Shen et al., 2020a). The positioning of the LayerNorm outside of the residual block causes gradients to shrink as they pass back through the network (Xiong et al., 2020a). In addition, the relative weight of the residual connections and main self-attention mechanism varies as we move through the network upon initialization (see figure 11.6c). There is the additional complication that the gradients for the query and key parameters are smaller than for the value parameters (Liu et al., 2020), which necessitates the use of Adam. These factors interact in a complex way, making training unstable and necessitating learning rate warm-up.

There have been various attempts to stabilize training, including (i) a variation of FixUp called *TFixup* (Huang et al., 2020a) that allows the LayerNorm components to be removed, (ii) changing the position of the LayerNorm components in the network (Liu et al., 2020), and (iii) re-weighting the two paths in the residual branches (Liu et al., 2020; Bachlechner et al., 2021). Xu et al. (2021b) introduced an initialization scheme called *DTFixup* that allows transformers to be trained with smaller datasets. A detailed discussion can be found in Prince (2021b).

Applications in vision: ImageGPT (Chen et al., 2020a) and the Vision Transformer (Dosovitskiy et al., 2021) were both early transformer architectures applied to images. Transformers have been used for image classification (Dosovitskiy et al., 2021; Touvron et al., 2021), object detection (Carion et al., 2020; Zhu et al., 2020b; Fang et al., 2021), semantic segmentation (Ye et al., 2019; Xie et al., 2021; Gu et al., 2022), super-resolution (Yang et al., 2020a), action recognition (Sun et al., 2019; Girdhar et al., 2019), image generation (Chen et al., 2021b; Nash et al., 2021), visual question answering (Su et al., 2019b; Tan & Bansal, 2019), inpainting (Wan et al., 2021; Zheng et al., 2021; Zhao et al., 2020b; Li et al., 2022), colorization (Kumar et al., 2021), and many other vision tasks (Khan et al., 2022; Liu et al., 2023b).

Transformers and convolutional networks: Transformers have been combined with convolutional neural networks for many tasks, including image classification (Wu et al., 2020a), object detection (Hu et al., 2018a; Carion et al., 2020), video processing (Wang et al., 2018c; Sun et al., 2019), unsupervised object discovery (Locatello et al., 2020) and various text/vision tasks (Chen et al., 2020d; Lu et al., 2019; Li et al., 2019). Transformers can outperform convolutional networks for vision tasks but usually require large quantities of data to achieve superior performance. Often, they are pre-trained on enormous datasets like JRT (Sun et al., 2017) and LAION (Schuhmann et al., 2021). The transformer doesn't have the inductive bias of convolutional networks, but by using huge amounts of data, it can surmount this disadvantage.

From pixels to video: Non-local networks (Wang et al., 2018c) were an early application of self-attention to image data. Transformers were initially applied to pixels in local neighborhoods (Parmar et al., 2018; Hu et al., 2019; Parmar et al., 2019; Zhao et al., 2020a). ImageGPT (Chen et al., 2020a) scaled this to model all pixels in a small image. The Vision Transformer (ViT) (Dosovitskiy et al., 2021) used non-overlapping patches to analyze bigger images.

Since then, many multi-scale systems have been developed, including the SWin transformer (Liu et al., 2021c), SWinV2 (Liu et al., 2022), multi-scale transformers (MViT) (Fan et al., 2021), and pyramid vision transformers (Wang et al., 2021). The Crossformer (Wang et al., 2022b) models interactions between spatial scales. Ali et al. (2021) introduced cross-covariance image transformers, in which the channels rather than spatial positions attend to one another, hence making the size of the attention matrix indifferent to the image size. The dual attention vision transformer (DaViT) was developed by Ding et al. (2022) and alternates between local spatial attention within sub-windows and spatially global attention between channels. Chu et al. (2021) similarly alternate between local attention within sub-windows and global attention by subsampling the spatial domain. Dong et al. (2022) adapt the ideas of figure 12.15, in which the interactions between elements are sparsified to the 2D image domain.

Transformers were subsequently adapted to video processing (Arnab et al., 2021; Bertasius et al., 2021; Liu et al., 2021c; Neimark et al., 2021; Patrick et al., 2021). A survey of transformers applied to video can be found in Selva et al. (2022).

Combining images and text: CLIP (Radford et al., 2021) learns a joint encoder for images and their captions using a contrastive pre-training task. The system ingests N images and their captions and produces a matrix of compatibility between images and captions. The loss function encourages the correct pairs to have a high score and the incorrect pairs to have a low score. Ramesh et al. (2021) and Ramesh et al. (2022) train a diffusion decoder to invert the CLIP image encoder for text-conditional image generation (see chapter 18).

Problems

Problem 12.1 Consider a self-attention mechanism that processes N inputs of length D to produce N outputs of the same size. How many weights and biases are used to compute the queries, keys, and values? Assume that all three quantities are also of length D . How many attention weights $a[\bullet, \bullet]$ will there be? How many weights and biases would there be in a fully connected shallow network relating all DN inputs to all DN outputs?

Problem 12.2 Why might we want to ensure that the input to the self-attention mechanism is the same size as the output?

Problem 12.3* Show that the self-attention mechanism (equation 12.8) is equivariant to a permutation $\mathbf{XP}$ of the data $\mathbf{X}$, where $\mathbf{P}$ is a [permutation matrix](#). In other words, show that:

$$\mathbf{Sa}[\mathbf{XP}] = \mathbf{Sa}[\mathbf{X}]\mathbf{P}. \quad (12.18)$$

Appendix B.4.4
Permutation
matrix

Problem 12.4 Consider the softmax operation:

$$y_i = \text{softmax}_i[\mathbf{z}] = \frac{\exp[z_i]}{\sum_{j=1}^5 \exp[z_j]}, \quad (12.19)$$

in the case where there are five inputs with values: $z_1 = -3$, $z_2 = 1$, $z_3 = 100$, $z_4 = 5$, $z_5 = -1$. Compute the 25 derivatives, $\partial y_i / \partial z_j$ for all $i, j \in \{1, 2, 3, 4, 5\}$. What do you conclude?

Problem 12.5 Why is implementation more efficient if the values, queries, and keys in each of the H heads each have dimension D/H where D is the original dimension of the data?

Problem 12.6 BERT was pre-trained using two tasks. The first task requires the system to predict missing (masked) words. The second task requires the system to classify pairs of sentences as being adjacent or not in the original text. Identify whether each of these tasks is generative or contrastive (see section 9.3.7). Why do you think they used two tasks? Propose two novel contrastive tasks that could be used to pre-train a language model.

Problem 12.7 Consider adding a new token to a precomputed masked self-attention mechanism with N tokens. Describe the *extra* computation that must be done to incorporate this new token.

Problem 12.8 Computation in vision transformers expands quadratically with the number of patches. Devise two methods to reduce the computation using the principles from figure 12.15.

Problem 12.9 Consider representing an image with a grid of 16×16 patches, each represented by a patch embedding of length 512. Compare the amount of computation required in the DaViT transformer to perform attention (i) between the patches, using all of the channels, and (ii) between the channels, using all of the patches.

Problem 12.10* Attention weights are usually computed as:

$$a[\mathbf{x}_m, \mathbf{x}_n] = \text{softmax}_m \left[\mathbf{k}_\bullet^T \mathbf{q}_n \right] = \frac{\exp \left[\mathbf{k}_m^T \mathbf{q}_n \right]}{\sum_{m'=1}^N \exp \left[\mathbf{k}_{m'}^T \mathbf{q}_n \right]}. \quad (12.20)$$

Consider replacing $\exp \left[\mathbf{k}_m^T \mathbf{q}_n \right]$ with the dot product $\mathbf{g}[\mathbf{k}_m]^T \mathbf{g}[\mathbf{q}_n]$ where $\mathbf{g}[\bullet]$ is a nonlinear transformation. Show how this makes the computation of the attention weights more efficient.